

exemple du cours précédent : BD vols-réservations

Voici 3 schémas de relations :

- avions(No_AV, NOM_AV, CAP, LOC)
- pilotes(No_PIL, NOM_PIL, VILLE)
- vols(No_VOL, No_AV, No_PIL, V_d, V_a, H_d, H_a)

et voici les 3 tables correspondantes :

avions				pilotes		
No_AV	NOM_AV	CAP	LOC	No_PIL	NOM_PIL	VILLE
100	airbus	300	nice	1	laurent	nice
101	airbus	300	paris	2	sophie	paris
102	carav	200	toulouse	3	claire	grenoble

vols						
No_VOL	No_AV	No_PIL	V_d	V_a	H_d	H_a
it100	100	1	nice	paris	7	8
it101	100	2	paris	toulouse	11	12
it102	101	1	paris	nice	12	13
it103	102	3	grenoble	toulouse	9	11
it104	101	3	toulouse	grenoble	17	18

SQL comme LDD : Exemple

- avions(no_AV, NOM_AV, CAP, LOC)
no_AV INTEGER
NOM_AV VARCHAR(20)
CAP SMALLINT
LOC VARCHAR(15)
- pilotes(no_PIL, NOM_PIL, VILLE)
no_PIL INTEGER
NOM_PIL VARCHAR(20)
VILLE VARCHAR(15)
- vols(no_VOL, no_AV, no_PIL, V_d, V_a, H_d, H_a)
no_VOL VARCHAR(5)
no_AV INTEGER
no_PIL INTEGER
V_d VARCHAR(15) H_d DATETIME
V_a VARCHAR(15) H_a DATETIME

Exemple : Création de la table avions à partir du schéma :

avions(no_AV, nom_AV, CAP, LOC)

```
CREATE TABLE avions (  
  no_AV INTEGER  
    CONSTRAINT Cle_P_avions PRIMARY KEY,  
  NOM_AV VARCHAR(20),  
  CAP SMALLINT  
    CONSTRAINT Dom_CAP_avions CHECK (CAP ≥ 4),  
  LOC VARCHAR(15) );
```

```
vols(no_VOL, no_AV, no_PIL, V_d, V_a, H_d, H_a)
CREATE TABLE vols (
    no_VOL VARCHAR(5)
        CONSTRAINT Cle_P_vols PRIMARY KEY,
    no_AV INTEGER
        CONSTRAINT Ref_no_AV_vols REFERENCES avions,
    no_PIL INTEGER
        CONSTRAINT Ref_no_PIL_vols REFERENCES pilotes,
    V_d VARCHAR(15) NOT NULL,
    V_a VARCHAR(15) NOT NULL,
    H_d DATETIME,
    H_a DATETIME,
    CONSTRAINT C1_vols CHECK (v_d ≠ v_a),
    CONSTRAINT C2_vols CHECK (h_d < h_a) );
```

INSERT :

- La commande INSERT permet d'insérer une ou plusieurs lignes de données dans une table selon la syntaxe suivante :

```
INSERT INTO nom_table(colonne1,colonne2,...)
VALUES (valeur1,valeur2,...);
```

Il faut noter que :

- Le nombre de colonnes et de valeurs doit être le même. De plus, les positions des colonnes doivent correspondre aux positions de leurs valeurs.
- Pour les colonnes qui ne figurent pas dans la liste spécifiées MySQL insert les valeurs par default ou alors NULL si elles ne sont pas spécifiées.
- Si une colonne est définie comme une colonne AUTO_INCREMENT, MySQL génère un entier séquentiel chaque fois qu'une ligne est insérée dans la table.

INSERT : Insérer plusieurs lignes

- Pour insérer plusieurs lignes dans une table à l'aide d'une seule instruction **INSERT**, on utilise la syntaxe suivante :

```
INSERT INTO nom_table(c1,c2,...)
VALUES (v01,v02,...),
(v11,v22,...),
..
(vn1,vn2,...)
;
```

INSERT : Exemples

- Soit la table « Produit » créée à partir de la requête suivante :

```
CREATE TABLE Produits (  
    Num_Produit VARCHAR(18) PRIMARY KEY,  
    description VARCHAR(40) DEFAULT 'Non specifié',  
    cout DECIMAL(10,2) NOT NULL CHECK (cout >= 0),  
    prix DECIMAL(10,2) NOT NULL CHECK (prix >= cout),  
    Date_ajout DATE  
);
```

- 1 - On veut ajouter le produit suivant : Numéro du Produit est P12, Son prix est 14 et le cout
- Voici la commande à exécuter :

```
INSERT INTO Produits(num_produit,cout,prix)  
VALUES ('P12',12,14);
```

INSERT : Exemples

- Ajout de plusieurs lignes : On veut ajouter les produits suivant :
 - Numéro du Produit est P13, Le cout: 120, le prix 140, ajouté le 01/01/2022.
 - Numéro du Produit est P100, Description: Laptop, le cout: 120, le prix 140, ajouté aujourd'hui.
- Voici la commande à exécuter :

```
INSERT INTO Produits(num_produit,description,cout,prix,date_ajout)  
VALUES ('P13','','120,140','2022-01-01'),  
      ('P100','Laptop',5000,6000,CURRENT_DATE)  
;
```


UPDATE : Exemples

- Rappelons la table Produits :

```
mysql> select * from Produits;
```

Num_Produit	description	cout	prix	Date_ajout
P100	Laptop	5000.00	6000.00	2022-01-14
P12	Non specifie	12.00	14.00	NULL
P13		120.00	140.00	2021-01-01

```
3 rows in set (0.00 sec)
```

- Modifier la date d'ajout du produit P12 en 31/12/2021 :

```
UPDATE Produits
```

```
SET
```

```
    Date_ajout = '2021-12-31'
```

```
WHERE Num_Produit='P12';
```

UPDATE : Exemples

- Modifier les données de telle façon que Description = 'Non specifiée' et Prix = 1

```
UPDATE Produits
SET
    Description = 'Non specifiée',
    Prix = 1.5*Coût ;
WHERE Num_Produit='P12' ;
```

- Résultat suivant les deux modifications :

```
mysql> select * from Produits;
+-----+-----+-----+-----+-----+
| Num_Produit | description | coût   | prix   | Date_ajout |
+-----+-----+-----+-----+-----+
| P100        | Non specifiée | 5000.00 | 7500.00 | 2022-01-14 |
| P12         | Non specifiée | 12.00   | 18.00   | 2021-12-31 |
| P13         | Non specifiée | 120.00  | 180.00  | 2021-01-01 |
+-----+-----+-----+-----+-----+
```

DELETE :

- L'instruction DELETE permet de supprimer une ou plusieurs lignes d'une table en utilisant la syntaxe suivante :

```
DELETE FROM nom_table
WHERE Conditions;
```

- Cette instruction permet d'utiliser, en option, une condition pour spécifier les lignes à supprimer dans la clause WHERE, si cette dernière est omise, l'instruction DELETE supprimera toutes les lignes de la table.
- Pour une table qui a une contrainte de clé étrangère, lorsque vous supprimez des lignes de la table parent, les lignes de la table enfant peuvent aussi être supprimées automatiquement à l'aide de l'option **ON DELETE CASCADE**.

DELETE : Exemples

- De la table Produit :

```
mysql> select * from Produits;
```

Num_Produit	description	cout	prix	Date_ajout
P100	Non specifie	5000.00	7500.00	2022-01-14
P12	Non specifie	12.00	18.00	2021-12-31
P13	Non specifie	120.00	180.00	2021-01-01

- On veut supprimer les Produits dont le cout <=12 .

```
DELETE FROM Produits
WHERE cout<=12;
```

- Voici la table après l'exécution de cette requête

```
mysql> DELETE FROM Produits
-> WHERE cout<=12;
Query OK, 1 row affected (0.01 sec)

mysql> select * from Produits;
```

Num_Produit	description	cout	prix	Date_ajout
P100	Non specifie	5000.00	7500.00	2022-01-14
P13	Non specifie	120.00	180.00	2021-01-01

2 rows in set (0.00 sec)

SELECT :

- L'instruction **SELECT** permet de consulter les données et de les présenter triées et/ou regroupées suivant certains critères.
- L'instruction **SELECT** basique est comme suit :

```
SELECT [Liste_select]
FROM nom_table;
```

- Dans cette syntaxe, il faut :
 - Spécifier une ou plusieurs colonnes à partir desquelles on veut sélectionner des données après le mot-clé **SELECT** : Pour sélectionner toute les colonnes de la table, on utilise « **SELECT *** », Sinon on spécifie les noms des colonnes séparés par une virgule (,).
 - Spécifier le nom de la table à partir de laquelle on veut sélectionner des données après le mot-clé **FROM**.
- N.B** : Lors de l'exécution de l'instruction **SELECT**, MySQL évalue la clause **FROM** avant la clause **SELECT** :

FROM

SELECT

SELECT :

- Options de la Requête SELECT :
 - La requête SQL plus avancées prend la forme suivante :

```
SELECT [DISTINCT] Liste_Select
FROM Liste_Tables
WHERE Liste_Conditions_Recherche
GROUP BY Liste_regroupement
HAVING Liste_Conditions_regroupement
ORDER BY liste_Tri
```

- MySQL exécute cette requête dans cet ordre :



DISTINCT :

- DISTINCT** est une option qui permet de supprimer les lignes en double.

Left screenshot: Query: `SELECT description FROM dbtest.produits;` Results: description, Laptop, Non specife, Laptop.

Right screenshot: Query: `SELECT distinct description FROM dbtest.produits;` Results: description, Laptop, Non specife.

WHERE :

- WHERE définit la liste de conditions que les données recherchées doivent vérifier. La condition de recherche est une combinaison d'une ou plusieurs expressions utilisant l'opérateur logique **AND**, **OR** et **NOT**. L'instruction **SELECT** inclura toute ligne qui satisfait la condition de recherche dans le jeu de résultats.
- WHERE est aussi utilisé dans **UPDATE** ou **DELETE** pour spécifier les lignes à mettre à jour ou à supprimer.

Left screenshot: Query: `SELECT * FROM dbtest.produits WHERE Date_ajout >= '2022-01-14';` Results: Num_Produit, description, cout, prix, Date_ajout; P100, Laptop, 5000.00, 6000.00, 2022-01-14; P120, Laptop, 6000.00, 7000.00, 2022-01-14; NULL, NULL, NULL, NULL, NULL.

Right screenshot: Query: `SELECT * FROM dbtest.produits WHERE Date_ajout >= '2022-01-14' AND Cout = 6000;` Results: Num_Produit, description, cout, prix, Date_ajout; P120, Laptop, 6000.00, 7000.00, 2022-01-14; NULL, NULL, NULL, NULL, NULL.

Num_Produit	description	cout	prix	Date_ajout
P100	Laptop	5000.00	6000.00	2026-01-26
P12	Non specife	12.00	14.00	2025-12-31
P13		120.00	140.00	2021-12-31
P14	Non specife	12.00	14.00	NULL